

Smart Meter Kafka authentication using SASL/SCRAM

This documentation provides detailed instructions on how to configure and authenticate to a Kafka cluster using SASL/SCRAM with Java and Python clients. Each client will have access to two dedicated topics and will use a specific group to consume data.

1. Introduction

SASL/SCRAM (Simple Authentication and Security Layer/ Salted Challenge Response Authentication Mechanism) is a mechanism for authenticating clients in Kafka. This guide will walk you through the steps required to set up SASL/SCRAM authentication for Kafka with Java and Python clients. Each client will be configured to access two dedicated topics and consume data using a specific consumer group.

2. Prerequisites

Before you begin, ensure you have the following:

- Kafka client tools (e.g., Kafka CLI, Java client, Python client)
- Properly configured client.properties files

3. Kafka cluster configuration

- 4 brokers:
PROD: [b1-smdata.reteleelectrice.ro:9001](#), [b2-smdata.reteleelectrice.ro:9002](#), [b3-smdata.reteleelectrice.ro:9003](#), [b4-smdata.reteleelectrice.ro:9004](#)

PRE-PROD: [b1-r-smdata.reteleelectrice.ro:9001](#), [b2-r-smdata.reteleelectrice.ro:9002](#), [b3-r-smdata.reteleelectrice.ro:9003](#), [b4-r-smdata.reteleelectrice.ro:9004](#)

- The Kafka cluster can be accessed by only using this bootstrap URL, afterwards the Kafka client will request the metadata from brokers.

PROD: [smdata.reteleelectrice.ro:9000](#)

PRE-PROD: [r-smdata.reteleelectrice.ro:9000](#)

- The certificate used for each DNS (bootstrap URL and brokers) is signed with Digicert Public CA and should be already trusted by many libraries.

4. Kafka topics

Each supplier will have two dedicated topics, one for LPO (Load Profile Order - load curves) and another for DCS (Daily Closures - daily readings). The naming convention of topics contains the Tax Code (CUI - the company's unique identification code) and type (LPO/DCS), all lowercase. To consume data from the topics, the supplier will use its own dedicated consumer group (group_<CUI>), user (user_<CUI>) and password. The credentials needed to access the cluster will be provided by Kafka administrators to the supplier.

Example of topics and credentials for supplier with CUI RO1234567:

- Topic names: `ro1234567_dcs`, `ro1234567_lpo`
- Consumer group: `group_ro1234567`
- User: `user_ro1234567`, Password: `xxxxxxxxxxxx`

5. Configuring Kafka Clients

- Kafka CLI:

client.properties configuration file:

client.properties

```
# Kafka bootstrap URL address
bootstrap.servers=smdata.reteleelectrice.ro:9000

# Security protocol
security.protocol=SASL_SSL

# SASL mechanism
sasl.mechanism=SCRAM-SHA-512

# JAAS configuration for SASL/SCRAM
sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule required \
  username="user_ro1234567" \
  password="xxxxxxxxxxxx";
```

Kafka CLI command to consume topic data:

```
./bin/kafka-console-consumer.sh --bootstrap-server smdata.reteleelectrice.ro:9000 --consumer.cc
```

- Python

Library installation

```
pip install confluent-kafka
```

```

from confluent_kafka import Producer, Consumer, KafkaError

# Common configuration
bootstrap_servers = 'smdata.reteleelectrice.ro:9000'
sasl_mechanism = 'SCRAM-SHA-512'
sasl_plain_username = 'user_ro1234567'
sasl_plain_password = 'xxxxxxxxxxxx'
security_protocol = 'SASL_SSL'

# Consumer configuration
consumer_config = {
    'bootstrap.servers': bootstrap_servers,
    'security.protocol': security_protocol,
    'sasl.mechanism': sasl_mechanism,
    'sasl.username': sasl_plain_username,
    'sasl.password': sasl_plain_password,
    'group.id': 'group_ro1234567',
    'auto.offset.reset': 'earliest'
}
consumer = Consumer(consumer_config)

# Subscribe to two topics and consume messages
consumer.subscribe(['ro1234567_dcs', 'ro1234567_lpo'])

while True:
    msg = consumer.poll(1.0)
    if msg is None:
        continue
    if msg.error():
        if msg.error().code() == KafkaError._PARTITION_EOF:
            continue
        else:
            print(msg.error())
            break
    print(f"Received message from {msg.topic()}: {msg.value().decode('utf-8')}")

consumer.close()

```

▪ PySpark

```

from pyspark.sql import SparkSession
from pyspark import SparkContext, SparkConf

spark_conf = SparkConf()
context = SparkContext(conf=spark_conf)
spark = SparkSession(context).builder.appName(session_name).getOrCreate()

df = (
    spark.read.format("kafka")
        .option("kafka.bootstrap.servers", kafka_credentials["bootstrap_server"])
        .option("subscribe", topic)
        .option("kafka.security.protocol", kafka_credentials["security_protocol"])
        .option("kafka.sasl.mechanism", kafka_credentials["sasl_mechanism"])
        .option(
            "kafka.sasl.jaas.config",
            f'org.apache.kafka.common.security.scram.ScramLoginModule required username="{kafka_cred
        )
        .option("startingOffsets", "earliest")
        .option("endingOffsets", "latest")
        .load()
)

```

▪ Java

Consumer

```
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.apache.kafka.common.serialization.StringSerializer;

import java.time.Duration;
import java.time.temporal.ChronoUnit;
import java.util.Arrays;
import java.util.Properties;

public class KafkaClientExample {
    public static void main(String[] args) {
        Properties props = new Properties();

        // Common properties
        props.put("bootstrap.servers", "smdata.reteleelectric.ro:9000");
        props.put("security.protocol", "SASL_SSL");
        props.put("sasl.mechanism", "SCRAM-SHA-512");
        props.put("sasl.jaas.config", "org.apache.kafka.common.security.scram.ScramLoginModule");

        // Consumer specific properties
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "group_ro1234567");
        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);

        // Subscribe to two topics
        consumer.subscribe(Arrays.asList("ro1234567_lpo", "ro1234567_dcs"));

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(Duration.of(100, ChronoUnit.SECONDS));
            for (ConsumerRecord<String, String> record : records) {
                System.out.printf("Consumed message from topic %s: %s%n", record.topic(), record.value());
            }
        }
    }
}
```